Containerized Monitoring & ML Anomaly Detection

A Rookie's Complete Guide: Architecture, How it Works & Step-by-Step Usage

1. What is this Project? (High-Level Overview)

Imagine you run a popular online website. Thousands of users click buttons, load pages, and checkout every minute. Normally, if your server gets overloaded or crashes, traditional monitoring alerts you **after** the outage happens. This project builds an **intelligent monitoring and predictive anomaly detection system**. It continuously measures live performance metrics (request rates, error rates, latency) and uses an **AI / Machine Learning model (Isolation Forest)** to score system health in real-time, predicting risk levels (**Normal** ■, **Medium Risk** ■, **High Risk** ■) before a total failure occurs.

2. The 7 Key Components of the Architecture

Component	Port / Location	What it Does (Beginner Explanation)
Flask App Backend	Port 5000	The core web server processing API requests & generating Prometheus metrics.
ML Prediction Service	Port 5001	The AI microservice running Isolation Forest model to evaluate live anomaly scores.
React Dashboard	Port 5173	The developer Web UI with interactive sliders, presets, and live risk meters.
Prometheus	Port 9090	The monitoring database that scrapes metrics every 10-15 seconds.
Alertmanager	Port 9093	The notification engine triggering alerts when ML risk score exceeds thresholds.
Grafana	Port 3000	The visual analytics interface rendering charts for latency, traffic, and error rates.
Locust Load Tester	Port 8089	A tool to simulate 50+ fake users pounding the API to test stress limits.

3. How the AI / ML Anomaly Detection Works

The system evaluates **4 key features** simultaneously:
• **Request Rate (req/s)**: How many incoming requests hit the server per second.
• **HTTP Error Rate (err/s)**: How many 500 Server Errors occur per second.
• **P90 Latency (seconds)**: The response time for 90% of requests (e.g. 0.21s baseline).
• **Requests In-Progress**: Active requests currently being handled concurrently.

The **Isolation Forest ML Model** evaluates these 4 metrics together against normal baseline data. Instead of simple rules, it outputs a continuous **Anomaly Score (0.0 to 1.0)**:
• **0.00 to 0.49** $\rightarrow$ **Normal (Code 0)**: System operating safely.
• **0.50 to 0.74** $\rightarrow$ **Medium Risk (Code 1)**: Traffic load or error rate drifting from baseline.
• **0.75 to 1.00** $\rightarrow$ **High Risk (Code 2)**: Severe metric anomaly detected!

4. How to Run the Project (Step-by-Step)

Option A: Running Locally (Fastest & Easiest for Development)

Open 3 separate terminals in VS Code and run the following commands:

Terminal 1 (Flask App Backend) — *Starts backend API on http://localhost:5000*

```
python app/src/app.py
```

Terminal 2 (ML Prediction Service) — *Starts ML AI Engine on http://localhost:5001*

```
python ml_service/app.py
```

Terminal 3 (React Developer Dashboard) — *Launches Developer UI on http://localhost:5173*

```
cd frontend
npm run dev
```

Option B: Running Containerized with Docker Compose

Make sure Docker Desktop is running, open a terminal at project root, and execute:

```
docker-compose up --build
```

5. How to Use the React Developer Dashboard

Open **http://localhost:5173/** in your browser. You will see two main tabs:

Tab 1: ML Intelligence (Anomaly Simulator)

• **Header Status Indicators**: Shows if API is Online and ML Engine is Active.
• **Status Card**: Displays the large live status (**Normal**, **Medium Risk**, or **High Risk**).
• **Anomaly Score Meter**: A continuous progress bar showing the raw score between 0.00 and 1.00.
• **Preset Cards**: Click *Normal Baseline*, *Ramp Up Load*, *Traffic Spike*, or *Heavy Error Storm* to test instant load profiles.
• **Interactive Sliders**: Move sliders to test custom numbers. **Crucial Note**: After moving sliders, click the **'Evaluate Custom Inputs'** button to send values to the ML service!

Tab 2: API Tester

Click endpoint buttons to test live HTTP responses:
• /health: Returns 200 OK (Healthy).
• /process: Simulates short data processing task.
• /slow: Simulates delayed 2-second response (used to test latency graphs).
• /error: Deliberately triggers a 500 server error (used to test error alerts).
• /metrics: Returns raw Prometheus metric exposition text.

6. Rookie Experiments to Try Right Now!

Experiment 1: How to Trigger HIGH RISK Status in Dashboard

1. Go to ML Intelligence tab in http://localhost:5173.

2. Set all 4 sliders to zero (Request Rate = 0, Error Rate = 0, P90 Latency = 0.01s, In-Progress = 0).

3. Click 'Evaluate Custom Inputs'.

4. Result: Anomaly Score jumps to ~0.98 - 1.00 (HIGH RISK ■) because zero activity is extreme out-of-distribution anomaly!

Experiment 2: Simulate 50 Fake Users with Locust

```
1. Open a new terminal: cd load-testing/locust/scripts
2. Run Locust: locust -f locustfile.py --host=http://localhost:5000
```

3. Open http://localhost:8089 in browser, set 50 Users, spawn rate 5, and click Start Swarm.

4. Watch live request metrics flow through Flask, ML service, and Prometheus!
